

REPORT

QUIZ GAME APPLICATION

RAJA ALI MEHRBAN(SP25-BDS-036)

TAHA ASHRAF SHAH BAZMI(SP25-BDS-046)

PROGRAMMING FUNDAMENTALS

MR.KHURRAM IQBAL

FINAL PROJECT

QUIZ GAME APPLICATION

MEMBERS: RAJA ALI,TAHA BAZMI

INTRODUCTION:

A quiz game in which you have to choose a subject of your own choice alongwith its difficulty level.Scoring will be done based on rules of the games.You can change subjects or quit after any question.

TASK DISTRIBUTION:-

ALI	TAHA
MENU CREATION	Setting game rules and asking input of subject and difficulty
TIMER	countQuestion function
CONDITIONS	displayQuestion function
getFile function	getPoints function
readQuestion function	getPenalty function
askAnswer function	saveScore function
Asking random question and making sure no question is repeated	Validate subject and difficulty choice
Main game loop	Report writing

FEATURES

KEY CONCEPTS USED:-

- FILE HANDLING
- EXCEPTION HANDLING
- 2D ARRAYS
- FUNCTIONS
- LOOPS
- CONDITIONS

ADDITIONAL INFORMATION:-

- MENU created showing different options
- Different subjects added each with different difficulty levels(easy,medium,hard)
- Created different files of each subject with its specific difficulty level
- Each file consists of 50 mcqs

WORKING OF OUR PROGRAM:-

The program starts with asking for user's name.After that rules of games are shown,the user has to score 50 points in order to win and if scores get below 0 then game over.You have to answer question within 15 seconds time frame.You can choose subject and difficulty of your own choice but scoring will be done based on difficulty criteria as shown in the game.

WORKING OF EACH FUNCTION:

1. `public static void main(String[] args)`

This is the main entry point of the program.

It initializes the game, handles the overall game flow, user input, subject/difficulty selection, and calls other functions.

It also manages win/loss conditions and game timer.

2. `public static String[] getFile()`

Prompts the user to select a subject and difficulty level from predefined options.

Validates the input and returns the chosen subject and difficulty as a string array.

Used to determine which question file to load.

3. `public static int countQuestions(File file)`

Reads through the file and counts how many questions are present.

Assumes each question is separated by a blank line.

Returns the total count of questions in the selected file.

4. `public static String[] readQuestions(File file, int count)`

Reads all the questions from the file and stores them in an array.

Each question is grouped and stored as one string entry.

Returns an array of question strings.

5. `public static void displayQuestion(String fullQuestion)`

Displays the question content line by line, excluding the line that contains the correct answer.

Helps ensure the answer is not shown to the player while asking the question.

Improves user experience during gameplay.

6. `public static String askAnswer(String question)`

Displays the question and prompts the user to answer within 15 seconds.
If time runs out, returns "T" to indicate timeout.
Validates that the input is A, B, C, or D.

7. public static void checkAnswer(String question, String userAnswer, String difficulty)

Checks if the user's answer matches the correct answer found in the question.
Updates the score based on correctness and selected difficulty.
Handles timeout cases and prints feedback to the user.

8. public static int getPoints(String difficulty)

Returns the number of points awarded for a correct answer based on the difficulty.
Easy = 1, Medium = 5, Hard = 10.
Used inside `checkAnswer()`.

9. public static int getPenalty(String difficulty)

Returns the number of points deducted for a wrong or timed-out answer based on difficulty.
Easy = 0, Medium = 5, Hard = 10.
Also used in `checkAnswer()`.

10. public static void saveScore(String status, long totalTime)

Saves the player's name, score, subjects played, game status (Win/Quit/Game Over), and total play time to `scores.txt`.
Writes over the file each time the game ends.
Used at all game-ending points.

CODE:-

```
import java.io.*;
import java.util.*;

public class LabProject {

    // Global variables to keep track of the player's name, score, and game
    history
    static int score = 0;
    static String name;
    static String subjectsUsed = "";
    static String difficultiesUsed = "";
    static Scanner input = new Scanner(System.in);

    public static void main(String[] args) {
        System.out.print("Enter your name: ");
        name = input.nextLine();

        System.out.println("\nWelcome, " + name + "!");
        System.out.println("\n Game Rules:");
        System.out.println("- You need 50 points to win the game.");
        System.out.println("- If your score drops below 0, you lose.");
        System.out.println("- Each question must be answered within 15
seconds.");
        System.out.println("- Scoring is based on difficulty:");
        System.out.println("    Easy:    +1 point, no penalty");
        System.out.println("    Medium: +5 points, -5 for wrong or timeout");
        System.out.println("    Hard:   +10 points, -10 for wrong or
timeout");
        System.out.println("- You can switch subjects or quit after any
question.");
        System.out.println("\nGood luck!\n");

        long startTimer = System.currentTimeMillis();

        // Main game loop
        while (true) {
            try {
                // Ask user for subject and difficulty
                String[] result = getFile();
                String subject = result[0];
                String difficulty = result[1];
                subjectsUsed = subjectsUsed + subject + " ";
                difficultiesUsed = difficultiesUsed + difficulty + " ";
                File myFile = new File(subject + difficulty + ".txt");

                int count = countQuestions(myFile);
                String[] questions = readQuestions(myFile, count);
                boolean[] askedIndexes = new boolean[count];
                int questionCounter = 0;

                while (true) {
                    // Winning condition
```

```

        if (score >= 50) {
            System.out.println("\nCongratulations, you won the
game!");

            long endTime = System.currentTimeMillis();
            long totalTime = (endTime - startTime) / 1000;
            saveScore("Win" , totalTime);
            return;
        }
        // Losing condition
        if (score < 0) {
            System.out.println("\nGame Over. Your score dropped
below zero.");

            long endTime = System.currentTimeMillis();
            long totalTime = (endTime - startTime) / 1000;
            saveScore("Game Over" , totalTime);
            return;
        }

        // Check if all questions in the file have been asked
        boolean allAsked = true;
        for (int i = 0; i < askedIndexes.length; i++) {
            if (!askedIndexes[i]) {
                allAsked = false;
                break;
            }
        }
        if (allAsked) {
            System.out.println("No more questions in this
file.");

            break;
        }

        // Select a random question that hasn't been asked
        int randomIndex;
        do {
            randomIndex = (int) (Math.random() * count);
        } while (askedIndexes[randomIndex]);

        askedIndexes[randomIndex] = true;
        String selectedQuestion = questions[randomIndex];
        String userAnswer = askAnswer(selectedQuestion);
        checkAnswer(selectedQuestion, userAnswer, difficulty);
        questionCounter++;

        // Check if score fell below zero after answering
        if (score < 0) {
            System.out.println("\n Game Over. Your score dropped
below zero.");

            long endTime = System.currentTimeMillis();
            long totalTime = (endTime - startTime) / 1000;
            saveScore("Game Over" , totalTime);
            return;
        }

        // Display score and present options to user
        System.out.println("\nScore: " + score);
        System.out.println("\nMenu:");

```

```

        System.out.println("1. Continue");
        System.out.println("2. Change Subject/Difficulty");
        System.out.println("0. Quit");
        System.out.print("Choose: ");

        int choice = -1;
        while (true) {
            try {
                choice = input.nextInt();
                if (choice >= 0 && choice <= 2) {
                    break;
                } else {
                    System.out.print("Invalid option. Please
enter 0, 1, or 2: ");
                }
            } catch (InputMismatchException e) {
                input.nextLine(); // Clear invalid input
                System.out.print("Please enter a valid number (0,
1, or 2): ");
            }
        }

        if (choice == 0) {
            long endTimer = System.currentTimeMillis();
            long totalTime = (endTimer - startTimer) / 1000;
            saveScore("Quit" , totalTime);
            return;
        } else if (choice == 2) {
            break; // Exits inner game loop to change
subject/difficulty
        }

    }
} catch (FileNotFoundException e ) {
    System.out.println("File not found.");
}
}

// Asks the user to choose subject and difficulty
public static String[] getFile() {
    String[] subjects = {"programming", "math", "biology"};
    String[] difficulties = {"easy", "medium", "hard"};
    int subjectChoice = -1;

    // Validate subject choice
    while (subjectChoice < 1 || subjectChoice > subjects.length) {
        System.out.println("Choose a Subject:");
        System.out.println("1. Programming\n2. Math\n3. Biology");
        try {
            subjectChoice = input.nextInt();
        } catch (InputMismatchException e) {
            input.nextLine();
            System.out.println("Please Enter a number.");
        }
    }
}

```



```

        // Validate difficulty choice
        int difficultyChoice = -1;
        while (difficultyChoice < 1 || difficultyChoice >
difficulties.length) {
            System.out.println("Choose Difficulty:");
            System.out.println("1. Easy\n2. Medium\n3. Hard");
            try {
                difficultyChoice = input.nextInt();
            } catch (InputMismatchException e) {
                input.nextLine();
                System.out.println("Please Enter a number.");
            }
        }

        return new String[]{subjects[subjectChoice - 1],
difficulties[difficultyChoice - 1]};
    }

    // Counts how many questions are in the file (each question separated by
a blank line)
    public static int countQuestions(File file) throws FileNotFoundException
    {
        Scanner Reader = new Scanner(file);
        int count = 0;
        while (Reader.hasNextLine()) {
            String line = Reader.nextLine();
            if (line == "") {
                count++;
            }
        }
        Reader.close();
        return count;
    }

    // Reads all questions from the file and stores them in an array
    public static String[] readQuestions(File file, int count) throws
FileNotFoundException {
        Scanner Reader = new Scanner(file);
        String[] questions = new String[count];
        String question = "";
        int index = 0;

        while (Reader.hasNextLine()) {
            String line = Reader.nextLine();
            if (line == "") {
                questions[index] = question;
                index++;
                question = "";
            } else {
                question = question + line + "\n";
            }
        }
        Reader.close();
        return questions;
    }

    // Displays the question without the answer line

```

```

public static void displayQuestion(String fullQuestion) {
    Scanner Reader = new Scanner(fullQuestion);
    while (Reader.hasNextLine()) {
        String line = Reader.nextLine();
        if (line.length() >= 9 && line.charAt(8) == ':') {
            continue;
        }
        System.out.println(line);
    }
    Reader.close();
}

// Prompts the user for an answer (A/B/C/D)
public static String askAnswer(String question) {
    displayQuestion(question);
    long startTime = System.currentTimeMillis(); //start time for
individual question
    String ans = "";
    while (true) {
        System.out.print("Enter your answer (A/B/C/D):[15 seconds to
Answer] ");
        ans = input.next().toUpperCase();

        long EndTime = System.currentTimeMillis();//ends time for
individual question
        long timeUsed = (EndTime - startTime) / 1000;
        if (timeUsed > 15){
            System.out.println("You took to long");
            return "T"; //We will use T as a timeout flag
        }
        if (ans.length() == 1) {
            char ch = ans.charAt(0);
            if (ch == 'A' || ch == 'B' || ch == 'C' || ch == 'D') {
                break;
            }
        }
        System.out.println("Invalid input. Please enter A, B, C, or D.");
    }
    return ans;
}

// Checks if the user's answer matches the correct one and updates score
public static void checkAnswer(String question, String userAnswer, String
difficulty) {
    if (userAnswer.length() == 1 && userAnswer.charAt(0) == 'T'){
        System.out.println("Answer not provided in time , You lost
points");
        score = score - getPenalty(difficulty);
        return;
    }
    String correctAnswer = "";
    Scanner Reader = new Scanner(question);
    while (Reader.hasNextLine()) {
        String line = Reader.nextLine();
        if (line == "") {
            continue;
        }
    }

```

```

        if (line.length() >= 13 && line.charAt(8) == ':') {
            char ch = line.charAt(line.length() - 3);
            correctAnswer = correctAnswer + ch;
        }
    }
    Reader.close();

    if (userAnswer.charAt(0) == correctAnswer.charAt(0)) {
        System.out.println("Correct!");
        score = score + getPoints(difficulty);
    } else {
        System.out.println("Incorrect. The correct answer is: " +
correctAnswer);
        score = score - getPenalty(difficulty);
    }
}

// Points based on difficulty
public static int getPoints(String difficulty) {
    switch (difficulty) {
        case "easy": return 1;
        case "medium": return 5;
        case "hard": return 10;
        default: return 0;
    }
}

// Penalties based on difficulty
public static int getPenalty(String difficulty) {
    switch (difficulty) {
        case "easy": return 0;
        case "medium": return 5;
        case "hard": return 10;
        default: return 0;
    }
}

// Saves the score and session info to a file
public static void saveScore(String status , long totalTime) {
    try {
        FileWriter output = new FileWriter("scores.txt"); // overwrite
file each time
        output.write("Name: " + name + "\n");
        output.write("Score: " + score + "\n");
        output.write("Subjects: " + subjectsUsed + "\n");
        output.write("Difficulties: " + difficultiesUsed + "\n");
        output.write("Status: " + status + "\n");
        output.write("Total Time Played " + totalTime + " seconds\n");
        output.close();
    } catch (IOException e) {
        System.out.println("Could not save score.");
    }
}
}

```

OUPUTS OF OVERALL PROGRAM:

```
D:\Programming Fundamentals\java\bin\java.exe -j
Enter your name: DataScientist

Welcome, DataScientist!

Game Rules:
- You need 50 points to win the game.
- If your score drops below 0, you lose.

- Each question must be answered within 15 seconds.
- Scoring is based on difficulty:
    Easy:  +1 point, no penalty
    Medium: +5 points, -5 for wrong or timeout
    Hard:  +10 points, -10 for wrong or timeout
- You can switch subjects or quit after any question.

Good luck!
```

This is the output display after taking input from user.

```
3. Biology

3
Choose Difficulty:
1. Easy
2. Medium
3. Hard
```

You choose the subject of your own choice.

```
3. Hard
2
47. What is the main function of sweat glands?
A. Produce oil
B. Excrete waste
C. Digest protein
D. Produce heat
Enter your answer (A/B/C/D):[15 seconds to Answer]
```

You choose difficulty of your own choice.

```
Enter your answer (A/B/C/D):[15 seconds to Answer] D
You took to long
Answer not provided in time , You lost points

Game Over. Your score dropped below zero.

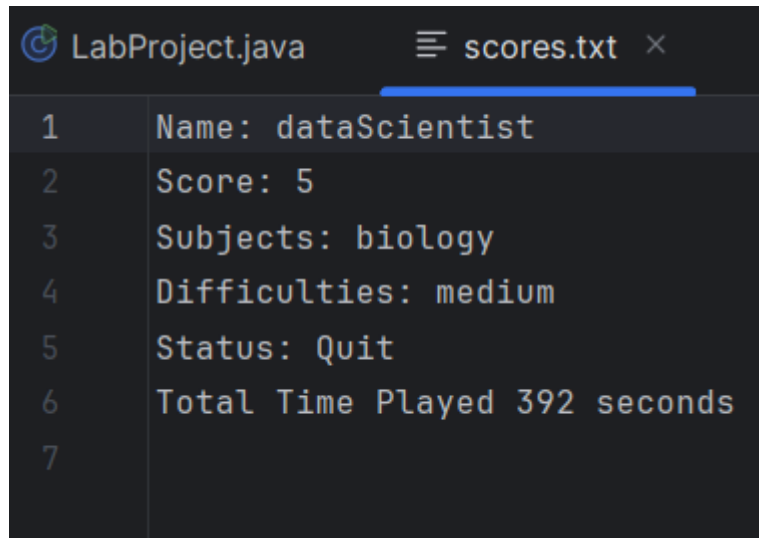
Process finished with exit code 0
```

If time exceeds 15seconds this display will be shown

```
Score: 5

Menu:
1. Continue
2. Change Subject/Difficulty
0. Quit
Choose: |
```

If answered correct points will be added based on difficulty criteria and then again its your choice whether to continue quit or change subject or its difficulty.



The screenshot shows a code editor with two tabs: 'LabProject.java' and 'scores.txt'. The 'scores.txt' tab is active and displays the following text:

```
1 Name: dataScientist
2 Score: 5
3 Subjects: biology
4 Difficulties: medium
5 Status: Quit
6 Total Time Played 392 seconds
7
```

At the end all your records will be saved in the file.